

Soyoung Yoo*
 Leekyo Jeong*
 Jinsu Ra
 Dongeon Lee

Cho Chun Shik Graduate School of Mobility,
 Korea Advanced Institute of Science and
 Technology,
 Daejeon 34051, Republic of Korea

Sunwoong Yang

Assistant Professor
 Department of Mechanical Engineering,
 Hanyang University,
 Ansan 15588, Republic of Korea

Hyogu Jeong

Narnia Labs,
 Daejeon 34051, Republic of Korea

Namwoo Kang¹

Associate Professor
 Cho Chun Shik Graduate School of Mobility,
 Korea Advanced Institute of Science and
 Technology, and Narnia Labs,
 Daejeon 34051, Republic of Korea
 e-mail: nwkang@kaist.ac.kr

DeepJEB++: Foundation Model-Driven Large-Scale 3D Engineering Dataset via 2D Latent Space Augmentation

Data-driven engineering design is constrained by the lack of large-scale 3D datasets that pair geometry with physics-based performance labels. In particular, existing 3D data augmentation techniques have limitations in preserving subtle and diverse geometric variations, and it remains difficult to automate the subsequent simulation-labeling process, where boundary conditions vary depending on the generated geometry. We present DeepJEB++, a foundation-model-driven data-augmentation framework that expands a small seed set of jet engine brackets into a large, simulation-labeled 3D dataset under constrained resources. Our key idea is to augment in the data-rich 2D latent space, then transfer to 3D. In Stage 1, we fine-tune a pretrained 2D latent diffusion model on multi-view renders and synthesize novel views by latent interpolation, retaining manufacturable designs through a vision-language-model (VLM) quality filter. In Stage 2, the validated images are lifted to 3D meshes by a domain-adapted generative foundation model. In Stage 3, an automated pipeline recognizes the load and bolt interfaces on each mesh and assigns finite-element labels—mass, stress, and displacement—without manual intervention. We assess augmentation quality along three intrinsic axes: manufacturability, label fidelity against the SimJEB ground truth, and distributional consistency. Starting from fewer than 400 seed designs, DeepJEB++ yields 15,360 simulation-labeled 3D brackets—a 40× expansion—using a single GPU per stage. The dataset will be made publicly available to support reproducible engineering-AI research.

Keywords: 3D dataset augmentation, foundation models, generative design, engineering data synthesis, latent space interpolation, vision-language model

1 Introduction

Data-driven methods are reshaping mechanical design, from generative design synthesis [1–3] to surrogate modeling of structural performance [4,5]. The effectiveness of these methods scales with the size and quality of the datasets that support them. Yet, unlike the natural-image domain—where billion-scale corpora are readily available—progress in engineering design is fundamentally constrained by the scarcity of large 3D datasets that pair geometry with physics-based performance labels. Such data demand expert CAD modeling and high-fidelity simulation, both of which are costly and difficult to scale.

This bottleneck is evident even in the most established benchmarks. The Simulated Jet Engine Bracket dataset (SimJEB) [6], one of the few public 3D engineering datasets, provides 381 hand-designed brackets with finite element analysis (FEA) results. DeepJEB [7] subsequently expanded this resource to 2,138 samples using a DeepSDF-based [8] generative model with an automated simulation pipeline. While these efforts mark important progress, their scale remains an order of magnitude below what modern deep architectures typically require, and each relies on a domain-specific generative model that is trained from scratch and must be rebuilt for every new domain, without access to the web-scale priors that now drive progress elsewhere. Two further limitations stem from how DeepJEB augments and labels its data. Because it augments *within* an implicit signed-distance field, the procedure tends to over-smooth the geometry and wash out the fine, locally varying features that distinguish individual brackets, narrowing the geomet-

ric diversity of the synthesized set relative to the original SimJEB designs. Moreover, to keep its labeling pipeline fully automatic, DeepJEB imposes a single, fixed boundary-condition template on every sample; this simplifies simulation but precludes a dataset that spans diverse boundary conditions.

In parallel, foundation models pretrained on web-scale data have begun to transfer well to engineering tasks, suggesting a path beyond hand-curated datasets. Harnessing this capability, however, has so far demanded enormous computational resources and access to proprietary weights [9], placing it out of reach for most academic and small-enterprise settings. This motivates a central question for democratizing data-driven design: *how can a small seed set be augmented into a large, simulation-labeled 3D dataset by adapting pretrained foundation models under severely limited resources?* We address this question with DeepJEB++, a large-scale jet engine bracket dataset built by adapting pretrained 2D and 3D foundation models under tightly constrained resources. Our key observation is that the asymmetry between 2D and 3D foundation models can be turned into an advantage: 2D generative models inherit priors from billions of images, whereas 3D models are trained on far smaller corpora. This leads to our central idea: rather than augmenting directly in the original 3D space—where generative priors are scarce—we perform the augmentation in the data-rich 2D image space and then lift the results back to 3D. DeepJEB++ realizes this idea by exploiting the 2D–3D asymmetry through a three-stage pipeline. Stage 1 augments a small seed set in the latent space of a fine-tuned 2D diffusion model and filters the synthesized images with a vision-language-model (VLM)-based quality classifier; in developing this classifier we identify and remedy a systematic negation artifact—which we term the *Negative Words Negation* (NWN) problem—that arises when VLMs describe the *absence* of

¹Corresponding Author.
 June 12, 2026

*S. Yoo and L. Jeong contributed equally to this work.

defects. Stage 2 lifts the validated images to 3D through domain-specific fine-tuning of a state-of-the-art 3D generative foundation model, which we show reconstructs reliably from even a single well-chosen viewpoint. Stage 3 closes the loop with an automated computer-aided engineering (CAE) pipeline that detects load and bolt interfaces on the generated meshes and assigns physics-based structural performance labels without manual intervention. Starting from fewer than 400 seed brackets, this pipeline produces an order-of-magnitude larger labeled 3D dataset using only a single GPU per training stage—demonstrating that domain-specific foundation-model pipelines, long assumed to require large-scale compute, are within reach of academic labs and small enterprises.

2 Related Work

2.1 3D Engineering Datasets. Public 3D datasets for engineering applications remain scarce compared to the computer vision domain. ShapeNet [10] and ABC [11] provide large collections of general 3D shapes but lack engineering performance labels. Domain-specific efforts include FRAMED [12] (4,500 parametric bike frames with FEM results), BIKED [13] (4,500 bicycle designs with machine-learning benchmarks), Ship-D [14] (ship hulls with drag coefficients), DrivAerNet++ [15] (car aerodynamics with CFD), and AircraftVerse [16] (27,714 multimodal aerial-vehicle designs).

For structural analysis, SimJEB [6] provides 381 jet engine brackets with FEA data, and DeepJEB [7] expanded this to 2,138 samples using a DeepSDF auto-decoder with automated simulation. However, both remain an order of magnitude smaller than modern architectures demand, and—more fundamentally—DeepJEB augments *within* a domain-specific implicit representation: a DeepSDF auto-decoder trained from scratch on the brackets themselves, so the augmentation inherits no prior beyond the seed set and each new domain requires retraining a bespoke generator. Beyond scale, this implicit augmentation sacrifices two kinds of diversity present in the original SimJEB designs—the fine geometric variation across hand-designed brackets and the per-design variation in load and bolt interfaces—and recovering both was the starting motivation for DeepJEB++. DeepJEB++ takes a different route. Rather than training a domain-specific generator, it *adapts* 2D and 3D foundation models pretrained on billions of images and hundreds of thousands of shapes, inheriting their external priors to expand the labeled dataset by an order of magnitude at single-GPU cost. It restores geometric diversity by augmenting in the data-rich 2D image space and recovers boundary-condition diversity by recognizing the load and bolt interfaces on each generated mesh, so that the constraints adapt to the individual geometry rather than following a single fixed template. Because the pipeline reuses general-purpose foundation models rather than a bracket-specific implicit network, its augmentation backbone is in principle, domain-agnostic, with cross-domain transfer left to future work.

2.2 Foundation Models for 3D Generation. The landscape of 3D generative models has rapidly evolved with the adoption of foundation model paradigms. Early image-to-3D systems such as Point-E [17] and novel-view diffusion (Zero-1-to-3 [18]), together with text-to-3D optimization (DreamFusion [19]), established conditional 3D generation, which recent foundation models have sharply improved in fidelity. TRELLIS [20] introduces Structured 3D Latents (SLAT), a unified sparse latent representation that supports decoding into multiple output formats (NeRF [21], 3D Gaussians [22], meshes) via rectified flow [23] transformers. Trained on over 500K 3D assets with up to 2 billion parameters, TRELLIS represents the current state-of-the-art in versatile 3D generation.

A parallel line of work targets fast feed-forward single-image-to-3D reconstruction: LRM [24] regresses a NeRF directly from one image, One-2-3-45 [25] lifts multi-view diffusion outputs to a mesh in seconds, Wonder3D [26] couples cross-domain (image and

normal) diffusion for reconstruction, and InstantMesh [27] pairs a multi-view diffusion model with a sparse-view LRM. Other notable models include TripoSG [28] (SDF-based with rectified flow) and Hunyuan3D [29] (high-quality but restricted licensing). We select TRELLIS for its MIT License, strong multi-view conditioning capabilities, and flexible output format—critical requirements for engineering applications where mesh quality directly impacts downstream simulation.

2.3 Data Augmentation via Generative Models. Generative data augmentation has been explored extensively in 2D [30,31] and increasingly in 3D [32,33]; diffusion models in particular have advanced performance-aware engineering design generation, such as for topology optimization [34]. However, most approaches operate within a single representation domain. Our work is distinguished by its **cross-dimensional transfer strategy**: we augment data in 2D latent space (where billion-scale pretrained knowledge exists) and then reconstruct to 3D (where domain-specific fine-tuning ensures geometric fidelity). This approach maximally leverages the knowledge embedded in large pretrained models while maintaining engineering validity through quality filtering. A data-augmentation method is judged by whether its samples are valid (on the data manifold), distributionally consistent with the seed set, and label-accurate; we assess DeepJEB++ against these criteria in Section 4.

2.4 Vision-Language Models for Quality Assessment. Building on contrastive vision-language pretraining [35], VLMs such as BLIP-2 [36] and LLaVA [37] have demonstrated strong visual understanding capabilities. We repurpose these models for a novel task: automated engineering quality assessment of synthesized images. Our approach differs from standard VLM applications in that it requires fine-grained detection of manufacturing defects (surface roughness, structural discontinuities) rather than high-level semantic understanding, motivating our choice of LLaVA’s spatial information-preserving architecture over BLIP-2’s text-centric compression.

3 Method

Figure 1 illustrates the overall DeepJEB++ pipeline, which consists of three main stages: 2D-based data augmentation (Stage 1), 3D mesh generation via foundation model fine-tuning (Stage 2), and boundary condition recognition with CAE analysis (Stage 3).

3.1 Domain Dataset: SimJEB. We use the SimJEB dataset [6] as our seed data source. SimJEB consists of 381 hand-designed jet engine bracket CAD models from the GE Jet Engine Bracket Challenge, with associated FEA simulation results. Following the data curation procedure of DeepJEB [7], we retain 380 geometrically stable samples with consistent boundary conditions as our seed set. Each bracket is characterized by a loaded interface and four bolted interfaces, imposing specific geometric constraints that generated designs must respect.

3.2 Stage 1: 2D-Based 3D Data Augmentation. The core insight of Stage 1 is that 2D image generation models, pretrained on billions of images, encode rich geometric priors that can be transferred to engineering domains through efficient fine-tuning. By operating in 2D latent space rather than directly in 3D, we can leverage this massive pretrained knowledge while sidestepping the data scarcity problem in 3D.

3.2.1 Multi-View Rendering. Each of the 380 SimJEB bracket meshes is rendered from 26 viewpoints (8 azimuth angles $\times$ 3 elevation angles + top/bottom views), producing a total of 9,880 multi-view images. Camera parameters (direction, focal distance, background type) are recorded in JSON format for each rendering, enabling precise view-conditioned generation in Stage 2. We standardize the rendering conditions (lighting, resolution, background) to ensure consistency across all samples.

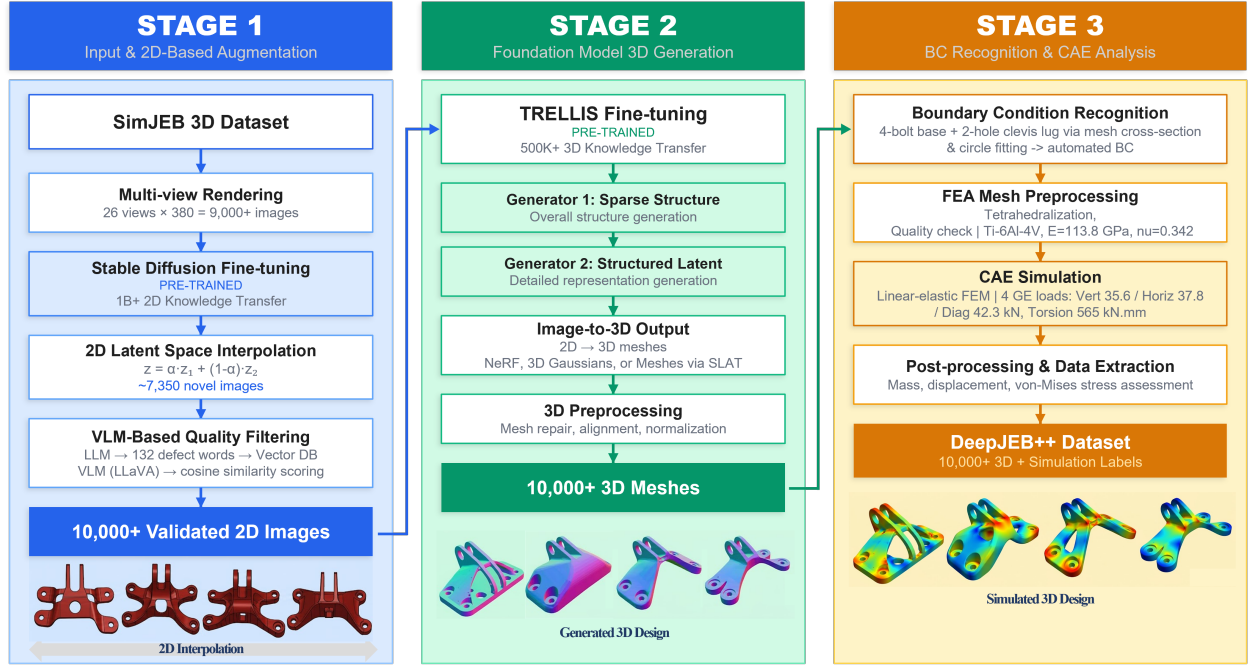


Fig. 1 Overview of the DeepJEB++ pipeline. Stage 1 augments 380 seed brackets into ~147,000 candidate 2D images via Stable Diffusion fine-tuning and latent-space interpolation, which VLM-based quality filtering reduces to 22,495 validated designs. Stage 2 lifts these to 3D meshes using a domain-adapted TRELLIS model. Stage 3 performs automated boundary-condition recognition and CAE simulation, producing the final DeepJEB++ dataset of 15,360 simulation-labeled 3D geometries.

3.2.2 Stable Diffusion Fine-Tuning. We fine-tune Stable Diffusion [38], a latent diffusion model (building on denoising diffusion probabilistic models [39]) pretrained on over 1 billion image-text pairs, on 7,800 SimJEB multi-view images (a 300-bracket training subset of the 380, rendered from 26 viewpoints each) using full parameter tuning. The text prompt “A mechanical bracket design.” is used as the conditioning signal. Training is conducted on a single A100 GPU for 1–2 days, yielding a model that generates bracket-specific geometries while retaining the general shape priors from pretraining.

This fine-tuning strategy is critical: without it, the model generates generic mechanical parts that lack the specific topological features of jet engine brackets. With fine-tuning, the model consistently produces designs that respect the characteristic multi-interface structure of the bracket domain.

3.2.3 Latent Space Interpolation. We generate novel bracket designs by interpolating between pairs of images [40] in the latent space of the fine-tuned Stable Diffusion model. Given two bracket images I_1 and I_2 , we encode them into latent vectors $\mathbf{z}_1$ and $\mathbf{z}_2$, and compute:

$$\mathbf{z}_{\text{interp}} = (1 - \alpha) \mathbf{z}_1 + \alpha \mathbf{z}_2 \quad (1)$$

where the mix ratio α is swept from 0 to 1 over 19 evenly spaced steps (IS00–IS18, including the two endpoints) along each interpolation path. After adding noise, the interpolated latent is decoded by the fine-tuned model with the text prompt condition; Figure 2(a) summarizes this encode–blend–noise–decode pipeline.

3.2.4 Representative Pair Selection. Naïve random pairing would produce many redundant interpolations between geometrically similar brackets. To ensure diversity, we apply the following selection procedure:

- (1) Convert all 380 bracket images to grayscale and normalize pixel intensities.
- (2) Apply Principal Component Analysis (PCA) for dimensionality reduction.

- (3) Score each sample by Euclidean distance in the reduced space.
- (4) Select 50 representative samples via 1D equal allocation across the distance distribution.

This yields $\binom{50}{2} = 1,225$ possible image pairs, of which 1,184 are generated; interpolating each pair across the mix-ratio steps above, with 8 rendered views per step, yields a pool of $\approx 147,000$ candidate 2D images before quality filtering.

3.2.5 Effect of Fine-Tuning on Interpolation. Domain fine-tuning is essential for the interpolation to stay within the bracket domain. Figure 2(b) compares interpolations between the same two SimJEB brackets—shown as seven evenly spaced frames of the 19-step path—decoded by the vanilla Stable Diffusion v1.5 model and by our SimJEB-fine-tuned model. The fine-tuned model yields valid, smoothly varying bracket geometries along the entire interpolation path, whereas the vanilla model collapses to generic mechanical parts (gear- and turbine-like structures) that violate the multi-interface bracket topology. Each panel—including the endpoints ($\alpha = 0, 1$)—is re-synthesized by the model from the interpolated latent through image-to-image generation, so every panel reflects the model’s own generative prior rather than a direct copy of the input; consequently the vanilla model distorts even the endpoints. This is not an encoding artifact: a pure VAE encode–decode round-trip of the endpoints is near-lossless, so the distortion stems from the bracket-unaware vanilla prior, not from the latent embedding.

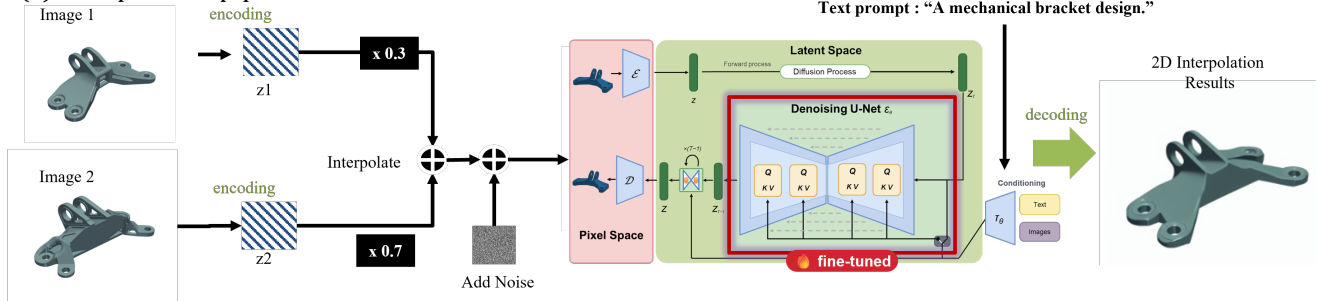
3.2.6 VLM-Based Quality Classifier. Not all synthesized images represent valid, manufacturable bracket designs. We develop an automated quality filtering framework using Vision-Language Models to assess each candidate image against engineering criteria.

3.2.7 Architecture. The classifier consists of three components (Figure 3):

- (1) **LLM branch:** Given engineering criteria (manufacturability, surface clearance, surface roughness, mono-body structure,

Stable diffusion latent interpolation

(a) Interpolation pipeline



(b) Interpolation comparison : vanilla vs fine-tune



Fig. 2 Stage-1 latent-space interpolation and its dependence on domain fine-tuning (Section 3.2.3). (a) *Interpolation pipeline*: two seed bracket images are encoded, latent-blended, and decoded by the fine-tuned diffusion model. (b) *Interpolation comparison*: one interpolation path ($\alpha = 0 \rightarrow 1$) decoded by the vanilla Stable Diffusion v1.5 model (*Vanilla SD* row) and our SimJEB-fine-tuned model (*Fine-tuned SD* row).

smooth surface, good strength), a large language model generates a set of 132 *negative words* describing potential defects (e.g., “scratches,” “cracks,” “poor adhesion”). These words are embedded into a vector database.

- (2) **VLM branch:** Each synthesized image is processed by a VLM with a prompt requesting description of the surface and structural characteristics. The VLM outputs a textual description of the image.
- (3) **Scoring:** The cosine similarity between the VLM description embedding and the negative word embeddings is computed; higher similarity indicates more defects. An image is filtered out when its score lies in the most defect-like top- $p\%$, with the threshold p calibrated on a labeled benchmark (Section 4.1).

3.2.8 Negative Words Negation (NWN) Problem. We identify a subtle failure mode in VLM-based quality assessment. When a VLM describes a high-quality image, it may use phrases like “no scratches” or “not rough.” These descriptions contain negative words that match the defect vocabulary, causing the similarity score to incorrectly penalize high-quality images. We term this the *Negative Words Negation (NWN) problem*.

We resolve this by adding an explicit constraint to the VLM prompt: “Do not use the word ‘no’.” This forces the VLM to use affirmative descriptions (e.g., “the surface is smooth” instead of “no scratches”), eliminating the double-negation artifact. This simple intervention improves classifier accuracy from 56.45% to **72.17%** on the labeled benchmark—the 636 decisive images among 900 manually labeled samples (the borderline “acceptably corrupt” class is excluded; Section 4.1).

3.2.9 VLM Model Selection. We compare two VLM architectures for the quality assessment task:

- **BLIP-2 [36]:** Uses a Q-Former for text-centric information compression. The heavy compression step risks losing fine-grained local features (surface roughness, micro-cracks) that are critical for engineering quality assessment.
- **LLaVA [37]:** Uses a projection matrix to map visual features directly to the language model, preserving patch-level spatial information. This architecture retains local defect information more effectively.

We select LLaVA for our pipeline due to its superior preservation of local geometric features, which is essential for detecting surface-level manufacturing defects.

The VLM filter is applied per candidate image; from the validated frames we then lift the diagonal view of each interpolation step across the 1,184 retained pairs to 3D, yielding **22,495 DeepJEB++ designs** from the ~147,000-image candidate pool.

3.3 Stage 2: 3D Generation via Foundation Model Fine-Tuning.

3.3.1 Model Selection. We evaluate three candidate image-to-3D foundation models: Hunyuan3D 2.0 [29], TripoSG [28], and TRELIS [20]. Licensing is a decisive criterion, since we intend to release the dataset openly and the generated geometries inherit the generator’s license: Hunyuan3D 2.0 carries regional usage restrictions (e.g., EU and Korea), and TripoSG relies on restrictively licensed dependencies, whereas TRELIS is distributed under the permissive MIT License. We therefore select TRELIS, which additionally supports multi-view conditioning and decodes its structured latents (SLAT) into multiple 3D formats (NeRF, 3D Gaussians, and meshes).

3.3.2 TRELIS Architecture. TRELIS employs a two-generator architecture operating on Structured 3D Latents (SLAT):

- **Sparse Structure Generator (SS-Generator):** Predicts which voxels in a 3D grid are active (i.e., where geometry exists), conditioned on input images. Uses a rectified flow transformer.
- **Structured Latent Generator (SLAT-Generator):** Fills the active voxels with detailed shape and appearance features, also based on a sparse flow transformer architecture.

This decomposition enables efficient generation by operating only on sparse, occupied regions of 3D space, substantially reducing computational cost compared to dense volumetric approaches.

3.3.3 Domain-Specific Fine-Tuning. We fine-tune both generators of TRELIS on the SimJEB dataset to adapt the pretrained model (originally trained on general 3D assets) to the engineering bracket domain.

3.3.4 Data Preparation Pipeline. For each of the 380 SimJEB meshes:

- (1) Render 150 multi-view images and extract DINO [41] visual features for each view.
- (2) Voxelize the 3D mesh to obtain sparse structure encoding.
- (3) Combine DINO features with sparse structure encoding to produce SLAT encoding.

The full data preparation requires approximately 10 hours. We use 300 samples for training and 80 for testing.

3.3.5 Training Protocol.

- **Generator 1 (SS-Generator):** Fine-tuned on 25 multi-view images per sample (a subset of the 150 rendered views) with corresponding sparse structure labels.
- **Generator 2 (SLAT-Generator):** Fine-tuned with single-image conditioning to generate detailed 3D representations from the structure predicted by Generator 1.

3.3.6 Inference and 3D Preprocessing. The fine-tuned TRELIS model converts validated 2D bracket images from Stage 1 into 3D meshes via SLAT decoding into NeRF, 3D Gaussians, or triangle meshes. The model supports both single-view and multi-view inference:

- **Single-view:** Each 2D synthesized image directly produces a 3D mesh.
- **Multi-view:** When multiple views of the same object are available, combining them improves reconstruction accuracy, saturating by about six views (Section 4.2); a single informative (diagonal) viewpoint already approaches this level.

After 3D generation, a preprocessing step performs mesh repair, alignment, and normalization to ensure geometric consistency across the generated dataset. This step corrects common artifacts from the image-to-3D conversion, such as non-manifold edges, self-intersections, and inconsistent orientation, producing clean triangle meshes suitable for downstream FEA preprocessing in Stage 3.

3.4 Stage 3: Boundary Condition Recognition and CAE Analysis. While Stages 1 and 2 produce 3D bracket meshes, engineering datasets require associated simulation labels for training surrogate models and evaluating structural performance. Stage 3 closes this gap through a fully automated pipeline that recognizes the load and bolt interfaces on each generated mesh, assigns physically consistent boundary conditions, and carries out Finite Element Analysis (FEA) to extract performance labels.

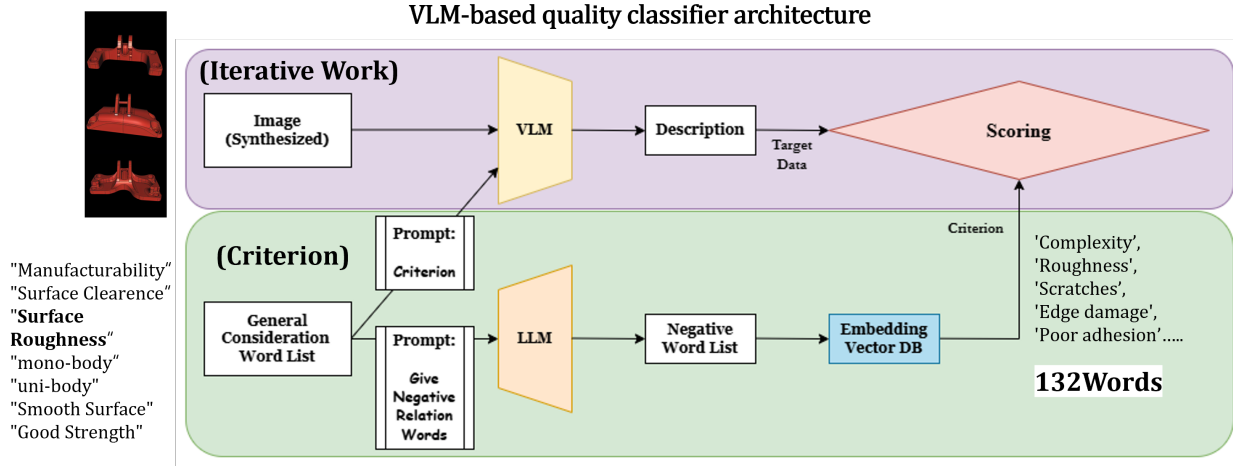


Fig. 3 VLM-based quality classifier architecture. The LLM generates domain-specific negative word embeddings, while the VLM produces image descriptions. Quality is assessed by the semantic distance between descriptions and defect vocabulary.

3.4.1 Automated Interface Detection. Bolt interfaces are detected directly on the mesh. Lower-flange vertices with normals perpendicular to the bolt axis are grouped into connected components via mesh adjacency rather than Euclidean clustering, which keeps adjacent bores separate and a single bore intact across a thick flange. A component is classified as a cylindrical bore when its vertex normals sweep $\geq 270^\circ$ about the axis and face inward. Coaxial components are merged, retaining only the smallest-diameter ring and discarding the larger counterbores. Selecting the outermost bore in each corner of the flanges yields the four clamped bolt interfaces.

The loaded clevis interface is identified by sectioning the mesh with planes perpendicular to the pin axis. Each prong bore then appears as a closed loop, to which a circle is fitted via the K sa method [42]. The loop centers are clustered in 3D, and the pair of equal radius bores separated by ≥ 20 mm is taken as the load-carrying two-hole clevis. For generated meshes, the four detected bolt centers are rigidly registered to a fixed SimJEB bolt template via Umeyama’s method [43]. The registration maps every mesh into the template’s global coordinate frame, keeping the applied boundary conditions consistent with the original SimJEB problem definition.

3.4.2 Geometric Validation of Detected Lugs. We validate each detected clevis with three pose-invariant geometric features computed from the fitted rings. The minimum angular coverage of the two bores, $\text{cov}_{\min}$, distinguishes a closed bore from a partial arc. The perpendicular offset of the two ring centers from the common pin axis, d_{XZ} , measures coaxiality. Their spacing along that axis, d_Y , is the clevis gap. A closed bore covers nearly the full circle at about 354° , whereas a partial arc spans only about 257° , so a candidate is accepted when $\text{cov}_{\min} \geq 330^\circ$, $d_{XZ} \leq 3$ mm, and $d_Y \geq 18$ mm.

We apply this rule to the full 381-bracket SimJEB set, which is disjoint from the 380-bracket augmentation seed. A separate manual review of the accepted candidates serves as the reference. Across 332 candidates, the rule matches this review on all but one borderline case, an agreement of 99.7%, keeping 294 genuine clevises and rejecting 38 spurious detections without any learned model. These 294 brackets, 77% of the 381, form the reference set for FEA validation.

Figure 4 shows the same detector applied to both real SimJEB CAD and generated DeepJEB++ meshes, detecting consistent four-bolt and lug-clevis interfaces on both. Generated meshes have thinner and slightly less coaxial interfaces, so the acceptance criterion is recalibrated on human labels (187 cases, about 99% agreement) and relaxed to $\text{cov}_{\min} \geq 300^\circ$, $d_{XZ} \leq 6$ mm, and $R \in [7, 11.5]$ mm.

Meshes that fail this criterion are flagged as invalid and excluded from FEA, so that only geometrically valid interfaces receive simulation labels.

3.4.3 Linear-Elastic Finite Element Solver. Each surface mesh is decimated to about 25,000 nodes without altering its topology, repaired into a single watertight surface, and tetrahedralized. The mesh quality is controlled by a radius–edge ratio below 1.2 and a minimum dihedral angle of 10° . Boundary-node ordering is preserved throughout, so that detected interface nodes retain their indices.

We solve linear elasticity, with bolt nodes fully clamped and the clevis interface subjected to a distributed nodal load, or a distributed nodal moment for the torsional case. Material properties and loads strictly follow the SimJEB and GE Jet Engine Bracket Challenge specification. The material is Ti–6Al–4V, with $E = 113.8$ GPa, $\nu = 0.342$, and a yield stress of 903 MPa (131 ksi at the service temperature, as specified by the GE Jet Engine Bracket Challenge). Four load cases are applied, a vertical load of 35.6 kN, a horizontal load of 37.8 kN, a diagonal load of 42.3 kN at 42° from vertical, and a torsional load of 565 kN-mm. Per element stresses are mapped to volume-weighted nodal von Mises values. For each load case we record the maximum nodal displacement and the von Mises stress. We report the 95th-percentile von Mises stress, σ_{vm}^{p95} , as the primary stress label, since it is robust to the single-point stress singularities that sharp generated edges can produce and that otherwise dominate the raw maximum. Together with the 3D geometries, these per-load displacement and stress labels constitute the simulation-annotated DeepJEB++ dataset.

4 Experiments

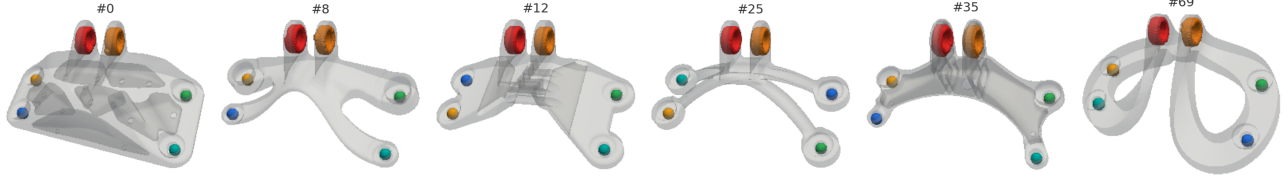
4.1 Stage 1: 2D Augmentation Quality.

4.1.1 Stable Diffusion Fine-Tuning Effect. Table 1 summarizes the fine-tuning configuration and qualitative effect. Without fine-tuning, Stable Diffusion generates generic mechanical parts. After fine-tuning on SimJEB multi-view images, the model consistently produces bracket-specific topologies with appropriate interface structures.

4.1.2 VLM Classifier Performance. Each candidate image receives a quality score equal to the cosine similarity between its LLaVA-generated surface description and a defect (negative-word) vocabulary; a higher score indicates a more defect-like description. Figure 5 summarizes the filter: the score distribution over all synthesized images with the $p=29\%$ cutoff, the confusion matrix on

Automated 4-bolt + lug-clevis detection: SimJEB vs DeepJEB++ brackets

(a) SimJEB — reference CAD



(b) DeepJEB++ — TRELIS-generated meshes

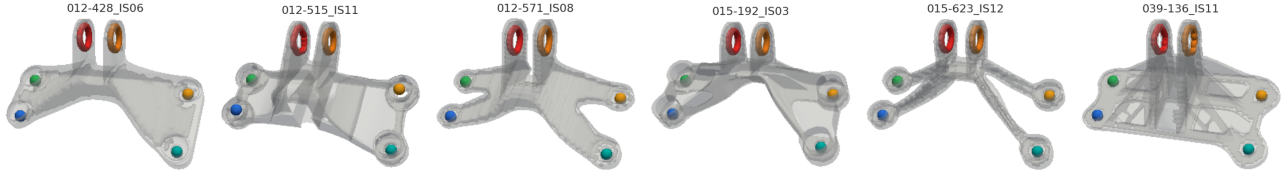


Fig. 4 Automated four-bolt and lug-clevis interface detection applied to (a) real SimJEB reference CAD and (b) DeepJEB++ TRELIS-generated meshes. The same geometric detector recovers consistent interfaces on both real and generated geometry, enabling identical boundary-condition assignment. Detected bolt centers are shown as colored markers and the two clevis bores as rings.

Table 1 Stage-1 configuration: Stable Diffusion fine-tuning and latent-space interpolation.

Component	Setting
<i>Backbone & fine-tuning</i>	
Base model	Stable Diffusion v1.5
Adapted module	Full U-Net (VAE, text encoder frozen)
Training images	7,800 SimJEB multi-view renders (512×512)
Text prompt	“A mechanical bracket design”
Optimizer	AdamW
Learning rate	1×10^{-5} (constant)
Batch size	16 (grad. accum. 1), fp32
Training steps	18,500
Compute	1× NVIDIA A100 (80 GB), ~1–2 days
<i>Latent-space interpolation (image-to-image)</i>	
Interpolant	$\mathbf{z}_{\text{mix}} = (1 - \alpha)\mathbf{z}_1 + \alpha\mathbf{z}_2$ (VAE latent)
Weights α	{0, 0.2, 0.4, 0.6, 0.8, 1.0}
Noise strength	0.8
Sampler / steps	UniPC [44] multistep / 20
Guidance scale	7.5 [45]
Seeds per pair	20

the labeled benchmark at that operating point, and representative retained brackets together with their LLaVA descriptions.

To quantify filtering quality, we construct a benchmark of 900 synthesized images manually labeled as non-corrupt (400, to be kept), severely corrupt (236, to be removed), and acceptably corrupt (264). The acceptably-corrupt class is borderline and subjective, so we exclude it from the accuracy computation, leaving 636 decisive samples. The filter removes an image when its quality score lies in the highest-scoring (most defect-like) top- $p\%$ and keeps it otherwise; we adopt a conservative removal rate of $p = 29\%$, discarding the most defect-like 29% of images. With the NWN-corrected prompt, accuracy on the labeled benchmark rises from 56.45% (vanilla prompt) to 72.17% at the default threshold, and at the adopted $p = 29\%$ reaches **76.10%** (false-positive rate 40.7%, false-negative rate 14.0%). Because p is a single fixed operating point rather than a per-sample-tuned quantity, the reported accuracy carries negligible selection optimism, which we confirm with two checks. First, on a larger labeled benchmark of 908 images (an expansion of the 636-sample set with additional frames), the filter attains 76.7% accuracy at the same $p=29\%$, essentially reproducing the value above and indicating that the oper-

ating point is not specific to one benchmark. Second, even when p is instead *selected* to maximize benchmark accuracy, repeated five-fold cross-validation yields a held-out accuracy of 81.8% against an in-sample optimum of 82.4%—a gap of only 0.6 percentage points—so choosing the removal rate on a labeled set introduces no material optimism. The filter is moreover deliberately operated as a *coarse first-stage screen*: its loose false-positive rate (40.7%) discards only the most defect-like images, while final geometric and physical validity is enforced downstream by the Stage-3 interface-detection and FEA gates (Section 3.4). The augmentation therefore does not depend on the VLM filter being precise.

4.1.3 Data Augmentation Summary. Starting from 380 SimJEB samples and 26 views each (9,880 images), our pipeline produces:

- 50 representative samples selected via PCA + Euclidean distance scoring
- 1,225 possible image pairs, of which 1,184 are generated
- ~147,000 candidate 2D images (8 rendered views per interpolation step)
- **22,495 DeepJEB++ designs** lifted to 3D in Stage 2 (diagonal view of each interpolation step, after VLM filtering)

4.2 Stage 2: 3D Generation Quality.

4.2.1 Single-View Reconstruction. We evaluate the fine-tuned TRELIS model on 80 held-out SimJEB test brackets using single-view inference. Table 2 shows that domain-specific fine-tuning substantially improves reconstruction quality, and Figure 6 confirms that this gain holds per bracket rather than only on average.

The pretrained TRELIS model, trained on general 3D assets, fails to capture the specific structural patterns of engineering brackets (multi-interface topology, load-bearing arm structures). Fine-tuning on only 300 SimJEB training samples is sufficient to recover these domain-specific features.

4.2.2 Multi-View Conditioning. We investigate how reconstruction quality depends on the input views—both their number and their viewpoint—on a held-out SimJEB bracket. Figure 7 reports Chamfer Distance for {1, 2, 3, 6, 10} input views (taken as contiguous chunks of the rendered viewpoints) and, for single-view inference, broken down by viewpoint. Chamfer Distance decreases as more views are added and saturates by about six views.

VLM (LLaVA, p11l) quality filtering: threshold optimization

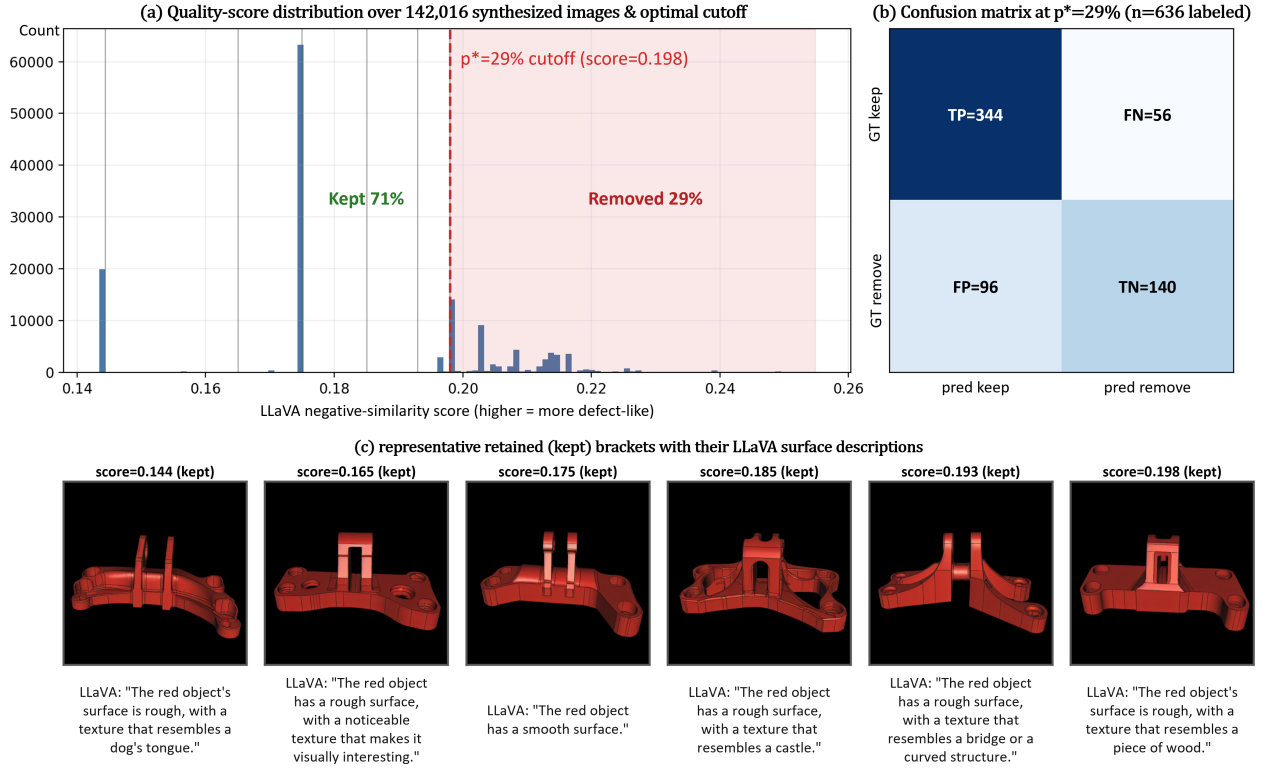


Fig. 5 LLaVA-based quality filtering. (a) Distribution of the quality score (cosine similarity between the LLaVA surface description and the defect vocabulary) over all synthesized images, with the adopted top- $p=29\%$ cutoff separating kept from removed designs. (b) Confusion matrix on the 636-sample labeled benchmark at that operating point. (c) Representative retained brackets and their LLaVA surface descriptions; the score is a negative-word similarity rather than a literal roughness axis, so retained brackets may still be described as “rough.”

Table 2 TRELLIS single-view inference results on SimJEB test set (80 samples).

Metric	No Fine-tuning	Fine-tuned	Improvement
IoU $\uparrow$	0.2390	0.4940	+106.69%
Chamfer Distance $\downarrow$	0.01199	0.00731	−38.98%

IoU vs Chamfer Distance (Resolution 64^3)

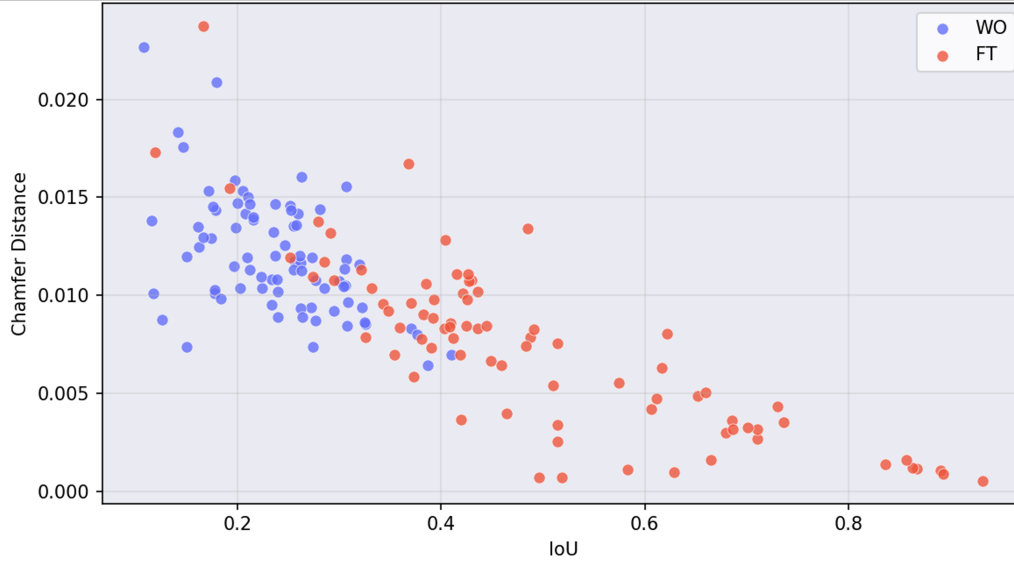


Fig. 6 IoU vs. Chamfer Distance (64^3 resolution) for the 80 test brackets. Reconstructions by the fine-tuned model (red, FT) attain higher IoU and lower Chamfer Distance than the pretrained baseline (blue, WO) for the large majority of brackets (means in Table 2), with the two distributions overlapping only in the mid-quality region—demonstrating the benefit of domain-specific adaptation.

Crucially, the *best* single view—a diagonal three-quarter view—already attains a Chamfer Distance (0.0031) essentially equal to the six-view result (0.0032), whereas degenerate viewpoints (top, front, bottom) are far worse (0.006–0.016). View *type* therefore matters more than view *count*: a single informative viewpoint suffices, and additional views mainly hedge against poor viewpoints.

This has a direct practical implication for our pipeline. Because the 2D designs synthesized by latent interpolation in Stage 1 have no pre-existing 3D object to re-photograph, multi-view conditioning is not available and large-scale generation necessarily relies on single-view inference. A well-chosen diagonal viewpoint matches the six-view reconstruction fidelity, so this choice incurs little loss; the fixed diagonal view used for generation is precisely the best-performing single viewpoint here.

4.2.3 2D Synthetic Image to 3D Generation. We demonstrate that the 2D synthesized images from Stage 1 can be successfully converted to 3D meshes by the fine-tuned TRELIS model. Figure 8(a) shows the end-to-end architecture: the interpolated 2D features condition the domain-adapted TRELIS model, which generates in two flow stages—sparse-structure generation followed by structured-latent generation (Section 3.3.2)—and decodes the structured latents into the output 3D representation (3D Gaussians, radiance fields, or meshes). Figure 8(b) shows the resulting meshes along one interpolation path (eight evenly spaced frames), illustrating that latent space interpolation in 2D translates to smooth, continuous shape variation in 3D.

4.3 Stage 3: Solver and Mass-Label Validation against SimJEB Ground Truth. Our dataset labels are the FEA results computed on the generated meshes themselves, not a reconstruction of any particular reference design. We accordingly validate the labeling pipeline, both its finite-element solver and its mass computation, on the real SimJEB meshes, where ground-truth values are available.

All 381 SimJEB CAD brackets carry ground-truth FEA labels. Here, 294 pass automated interface detection (Section 3.4.1), 289 yield a stable node-level fit, and 279 are single-body meshes with valid mass labels, which serve as the comparison reference (Sec-

tion 4.6). The 380-bracket augmentation seed (Section 3.1) is the geometrically valid subset of the 381.

To quantify the pipeline’s accuracy on a given mesh, we compare its output against the original SimJEB FEA results on the original brackets with validated interface detection (294 of 381 cases; Section 3.4.1), under the identical material and loading configurations.

A direct comparison of scalar peaks is confounded: the applied load direction is recovered from geometry, and the decimated meshes differ from the reference discretization, so peak stresses and displacements mix load-direction uncertainty with solver fidelity. We therefore isolate solver fidelity with a node-level test. Because the elastic response is linear (by superposition) in the interface load, we solve six unit responses at the loaded interface—three forces (F_x, F_y, F_z) and three moments (M_x, M_y, M_z)—and least-squares fit their six coefficients to the SimJEB ground-truth nodal displacement field. The resulting coefficient of determination R^2 measures how closely the two solvers agree once load-direction and magnitude uncertainty is absorbed, thereby reflecting pure discretization and boundary-condition fidelity. This six-degree-of-freedom basis matches the force-plus-moment load that SimJEB applies through an RBE3 reference point; a force-only (three-DOF) basis cannot reproduce the torsional case. Fitting a displacement field of tens of thousands of nodal degrees of freedom with only six coefficients ($6 \ll 3N$) is far from an over-parameterized fit.

Table 3 and Figure 10 report the node-level R^2 across the four load cases: the medians fall between 0.88 and 0.94, the bulk of brackets exceed 0.8 (88–91% for the in-plane and torsional loads, 69% for the harder diagonal case), and the best-conditioned bracket (#556) reaches 0.99–1.00 on every load. Figure 10(b) shows this agreement at the node level for a representative bracket (case 69), whose per-node displacements track the reference along the identity line ($R^2 = 0.996$, slope 1.00, $n = 13,204$ nodes). A small number of low-deformation cases (peak displacement < 0.3 mm) fall near the solver’s numerical noise floor and yield unstable R^2 , so we report the median rather than the mean. These results indicate that, up to load identification, the automated pipeline’s finite element solution is in close agreement with the reference SimJEB simulation. The residual gap from unity is attributable to the bolt-constraint boundary condition—our full clamp versus the

Multi-view inference results based on the number of input images

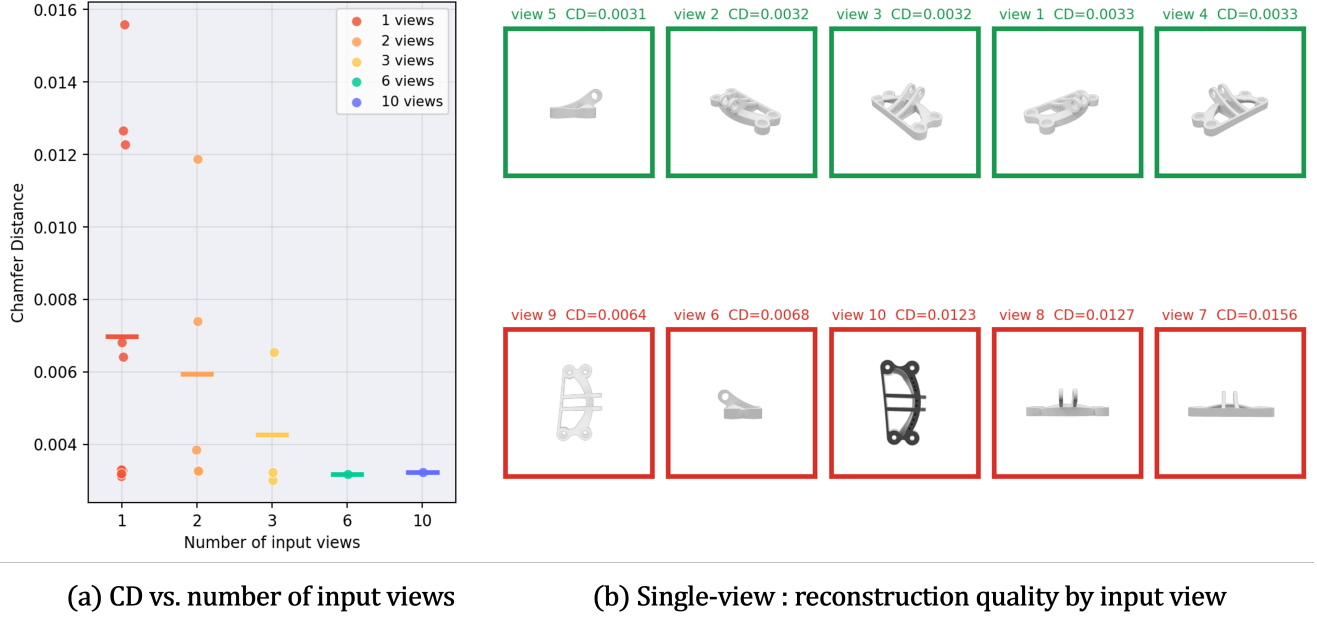


Fig. 7 Reconstruction quality versus input views on a held-out SimJEB bracket. (a) Chamfer Distance for {1, 2, 3, 6, 10} input views; the best single (diagonal) view already matches the six-view level. (b) Single-view reconstruction by viewpoint, sorted best to worst: diagonal three-quarter views (green) reconstruct well and are the viewpoint used for large-scale single-view generation, while top/front/bottom views (red) are degenerate.

2D-to-3D reconstruction examples

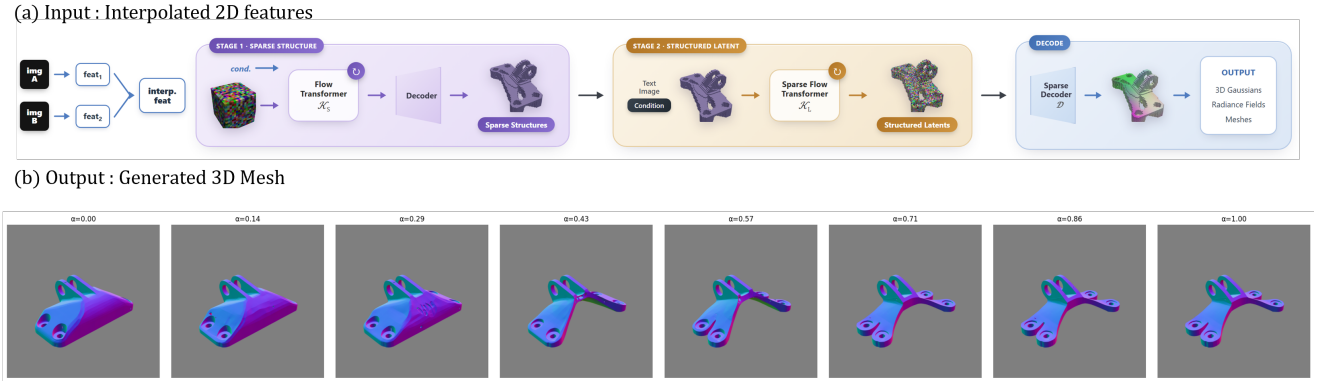


Fig. 8 End-to-end 2D-to-3D generation in DeepJEB++ (Section 3.3). (a) Input: interpolated 2D features (img A, img B encoded and blended into *interp. feat*) conditioning the domain-adapted TRELLIS model. (b) Output: generated 3D mesh along one interpolation path ($\alpha = 0 \rightarrow 1$).

Table 3 Accuracy of the automated CAE pipeline on the reference SimJEB brackets, where ground truth exists. *Top*: node-level displacement agreement from a six-DOF (three-force, three-moment) least-squares fit, over 289 of the 294 validated brackets (five near-zero-deformation cases excluded). *Bottom*: agreement of the computed mass label with the CAD ground truth.

	Vertical	Horizontal	Diagonal	Torsional
Median R^2	0.94	0.94	0.88	0.91
Interquartile range	0.88–0.97	0.89–0.97	0.76–0.96	0.86–0.94
Cases with $R^2 \geq 0.8$ (%)	88	90	69	91
Best case (#556)	0.998	0.998	0.994	0.994
Mass label (mesh volume $\times$ density) vs CAD GT: $R^2=0.9999$, median $ \text{err} =0.06\%$, 99% within 1%				

reference’s RBE2 bolt spider, which permits rotation about the bolt axis (distinct from the RBE3 point used for load application above)—and to mesh decimation. The mass label, computed as mesh volume times material density, is essentially exact: it matches the SimJEB CAD ground-truth mass with $R^2 = 0.9999$ and a median relative error of 0.06%, with 99% of the 279 single-body meshes within 1% (Table 3). Beyond these aggregate validation metrics, Figure 9 shows the spatial displacement and von Mises stress fields that the automated pipeline produces for a representative deployable bracket (case 545-364, whose four-load responses lie on the deployable-set medians) under the four load cases. The pipeline recovers physically consistent load paths: the loaded clevis and the lower web carry the in-plane vertical, horizontal, and diagonal cases, whereas the torsional case engages the part more uniformly at lower amplitude (peak $|u| = 2.48/2.20/1.05/0.98$ mm and $\sigma_{vm}^{p95} = 716/699/516/319$ MPa for the vertical/horizontal/diagonal/torsional cases, respectively). These per-load fields constitute the dataset labels. The fields are re-solved on a finer mesh for visualization, so their absolute magnitudes are mesh-resolution dependent; node-level fidelity is therefore established separately against the ground truth (Fig. 10), and dataset-level comparisons use self-normalized responses (Fig. 14).

4.4 Dataset Statistics. Table 4 compares DeepJEB++ with existing jet engine bracket datasets.

We summarize the end-to-end yield of the pipeline (Fig. 11). Of the 22,495 generated designs, 22,367 produced meshes that entered the automated FEA pipeline, of which 17,391 solved successfully. A single-body mesh-quality screen (largest connected-component volume fraction ≥ 0.95) then removes fragmented and residual multi-body meshes, leaving **15,360 deployable simulation-labeled brackets**—9,063 clean single-body meshes plus 6,297 from which a minor disconnected speck is removed, after conservatively excluding 39 further cases whose interfaces failed re-detection during automated mass computation. This is a 40 $\times$ expansion over the 381-bracket SimJEB seed set, produced under a single-GPU budget. The losses break down as 128 designs lost at mesh generation (22,495 $\rightarrow$ 22,367), and then, among the 7,007 meshes removed between the watertight-mesh and valid-label stages: no four-bolt interface (−2,428), the lug–clevis geometric gate (−1,350), no clevis detected (−334), FEM timeout (−31), and fragmented mesh/registration / other (−2,864).

4.5 Augmentation Coverage in the Latent Space. Finally, we examine how the augmented data relate to the seed set and to the prior DeepJEB dataset. We voxelize every bracket to a common 64^3 grid, align each shape to a canonical pose by its principal axes (so that orientation does not confound the comparison), and render it from the diagonal three-quarter viewpoint used for single-view generation (the best-performing view in Fig. 7); the renders are encoded with the Stable Diffusion VAE encoder and the resulting appearance embeddings are projected to two dimensions by PCA (Fig. 12). A useful augmentation should remain on the seed data manifold while broadening it, rather than drifting to a disjoint or degenerate region. We test this in a pose-normalized appearance space using all deployable samples (380 SimJEB, 2,138 DeepJEB, and 15,360 DeepJEB++; Fig. 12).² The three datasets overlap substantially: the SimJEB and DeepJEB samples fall almost entirely within the DeepJEB++ support, so DeepJEB++ covers and modestly extends the manifold spanned by both the seed set and the earlier DeepJEB augmentation rather than collapsing or drifting away.

This coverage reflects valid, on-distribution augmentation rather than unbounded geometric novelty. Part of the larger DeepJEB++ hull follows from its order-of-magnitude larger sample count: under a sample-balanced comparison (equal n per dataset) the three

convex-hull areas become comparable, and a nearest-neighbor test keeps the datasets separable above the 0.5 chance level even as they heavily overlap—consistent with augmented samples that are novel instances yet drawn from the seed distribution. The contribution of DeepJEB++ therefore, rests on augmenting the labeled set at scale with physically consistent labels (Section 4.3), not on geometric novelty per se.

4.6 Comparison with SimJEB: Mass and Structural Response. As a further augmentation-quality check, we test whether the augmented dataset is distributionally consistent with the SimJEB reference—beyond the per-bracket label fidelity established in Section 4.3—along two *mesh-robust* axes: part mass and the shape of the structural-response distribution. Mass is computed directly from the mesh volume and is insensitive to the resolution difference between the datasets—the computed mass matches the CAD ground truth to $R^2 = 0.9999$ (Fig. 10). The DeepJEB++ brackets are substantially lighter than SimJEB, with a median mass of 233 g versus 674 g (Fig. 13a), indicating that the generative pipeline explores a thinner, more compliant region of the design space. The mass–stress scatter (Fig. 13b,c) shows the expected inverse relation—lighter brackets carry higher peak stress—with DeepJEB++ following the same trend as SimJEB while extending the envelope toward lighter, more highly stressed designs.

Absolute displacement and stress magnitudes, by contrast, are mesh-confounded (SimJEB is distributed at $\sim 50\%$ decimation, whereas DeepJEB++ retains near-full meshes), so we compare only the *shape* of each response distribution after normalizing every sample by its own dataset median (Fig. 14). Across all four load cases, the self-normalized displacement and 95th-percentile von Mises distributions of DeepJEB++ track those of SimJEB, indicating that the automatically generated labels reproduce the reference response-distribution shape and are physically consistent.

5 Discussion

5.1 Foundation Model Knowledge Transfer. Our results suggest that pretrained foundation models encode geometric priors applicable to this engineering domain. The fine-tuned Stable Diffusion model’s ability to generate bracket-specific designs from a generic text prompt suggests that the model has captured appearance-level regularities of mechanical topology (load paths, structural connectivity) during pretraining on diverse images. Similarly, TRELIS’s rapid adaptation to the bracket domain (300 training samples) indicates that its pretrained 3D priors (learned from 500K+ shapes) transfer effectively to specialized engineering geometries.

5.2 Cross-Dimensional Augmentation. The three-stage 3D $\rightarrow$ 2D $\rightarrow$ 3D $\rightarrow$ simulation pipeline enables us to exploit the asymmetry between 2D and 3D foundation model maturity. While 2D generative models benefit from billion-scale pretraining datasets, 3D models are limited to hundreds of thousands of shapes. By augmenting in 2D latent space and reconstructing in 3D, we draw on web-scale 2D priors for generation while securing geometric fidelity through lightweight 3D domain adaptation—a route fundamentally distinct from training a domain-specific implicit generator (e.g., DeepSDF) from scratch. Because Stages 1–2 adapt general-purpose foundation models rather than a bracket-specific network, the augmentation backbone is domain-agnostic; re-targeting it to a new domain mainly requires adapting the Stage-3 labeling, and demonstrating cross-domain transfer is left to future work. The automated CAE pipeline in Stage 3 completes the augmented dataset with physics-based performance labels; quantifying the downstream benefit of this augmentation is likewise an important direction for future work.

5.3 VLM as Engineering Inspector. The VLM-based quality classifier offers a complementary, semantic approach to auto-

²This geometry-coverage analysis uses the full 380-bracket seed set; the labeled comparisons in Section 4.6 use the 279 single-body SimJEB brackets that additionally carry valid mass labels.

Automated FEA result fields on a representative DeepJEB++ bracket (case 545-364): four load cases

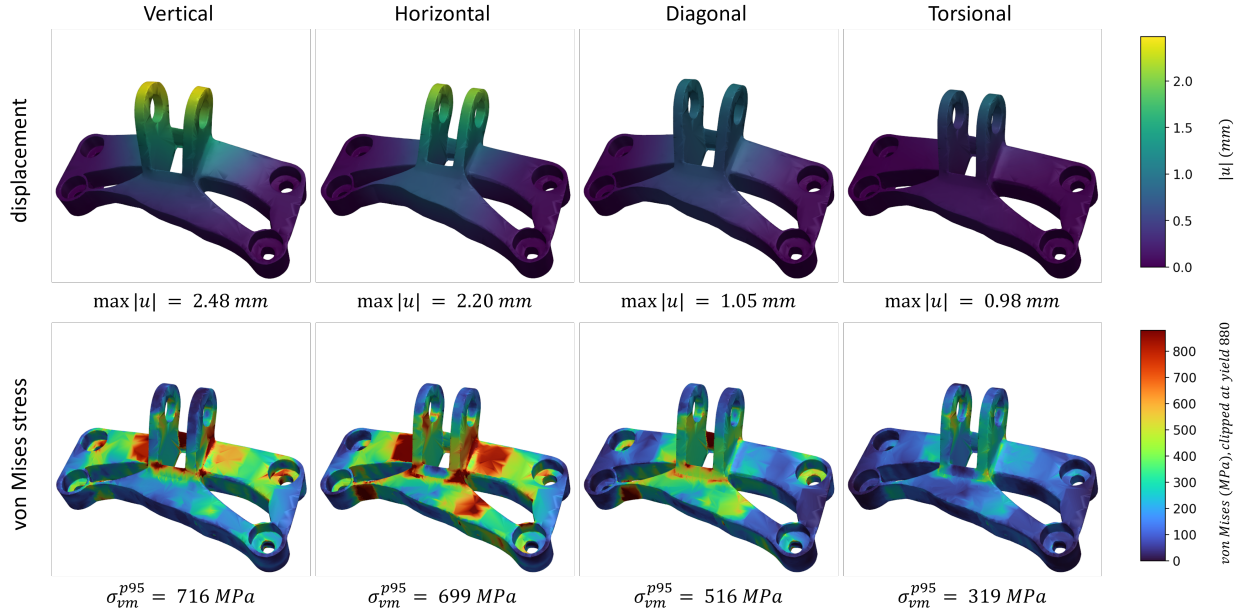


Fig. 9 Automated FEA result fields for a representative deployable DeepJEB++ bracket (case 545-364) under the four load cases (columns). *Top*: displacement magnitude $|u|$ on the deformed shape (9 $\times$ exaggerated). *Bottom*: nodal von Mises stress, clipped at the 903 MPa Ti-6Al-4V yield.

Node-level validation of the automated FEM against SimJEB ground truth (6-DOF least-squares fit)

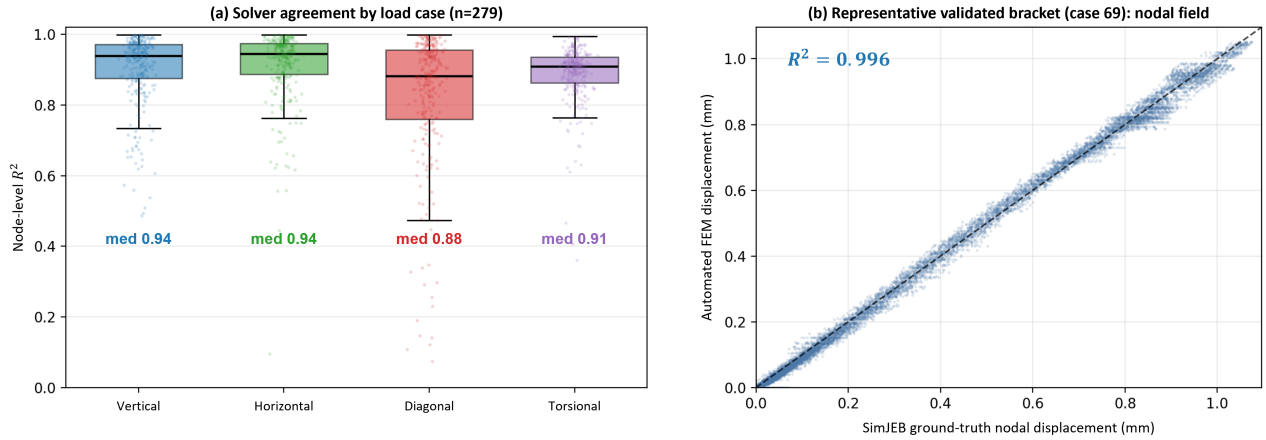


Fig. 10 Node-level validation of the automated FEM against the SimJEB ground truth (Section 4.3). (a) Per-bracket R^2 distribution by load case over the 289 validated brackets, medians annotated. (b) Per-node displacement for a representative bracket (case 69) against the reference field.

Table 4 Comparison of jet engine bracket datasets. Seed counts are as reported by each source: DeepJEB [7] used 263 SimJEB brackets, whereas DeepJEB++ uses 380 of the 381.

	SimJEB	DeepJEB	DeepJEB++
Seed data	—	263 (from SimJEB)	380 (from SimJEB)
Total 3D samples	381	2,138	15,360
Simulation labels	FEA (manual setup)	FEA (automated)	FEA (automated BC + CAE)
Expansion ratio	1 $\times$	5.6 $\times$	40$\times$
Generation method	Hand-designed	DeepSDF interpolation	Foundation model transfer
2D pretraining	None	None	Stable Diffusion (1B+)
3D pretraining	None	None	TRELLIS (500K+ shapes)
Quality filtering	Manual + IQR	Min Jacobian + IQR	VLM-based classifier

DeepJEB++ generation-to-label yield

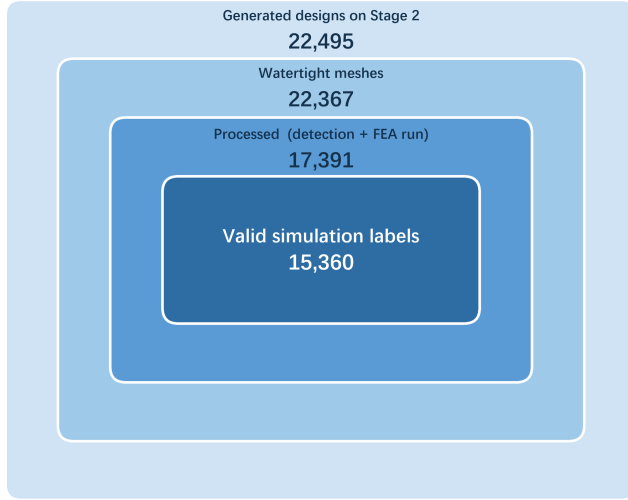


Fig. 11 Generation-to-label yield of the DeepJEB++ pipeline, from 22,495 generated designs to 15,360 deployable simulation-labeled brackets. Per-stage counts and losses are given in the text.

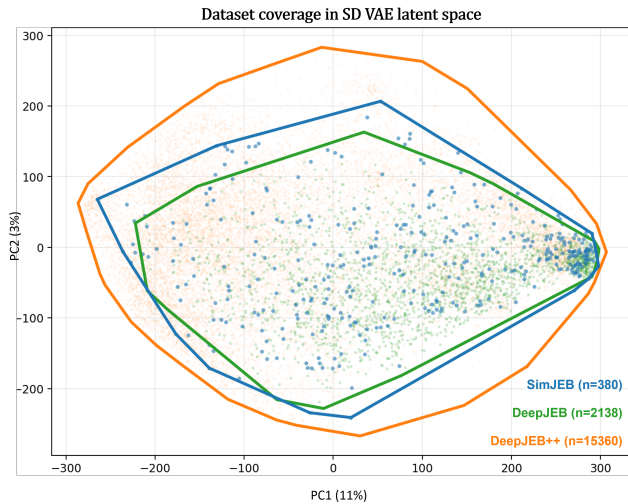


Fig. 12 Dataset coverage in a pose-normalized SD-VAE appearance space (SimJEB $n=380$, DeepJEB $n=2,138$, DeepJEB++ $n=15,360$; PCA projection of the appearance embeddings). The SimJEB and DeepJEB samples fall almost entirely within the DeepJEB++ (orange) support, indicating that the augmentation covers and modestly extends the manifold of both while staying on-distribution; the larger DeepJEB++ hull partly reflects its much larger sample size (see text).

mated engineering quality assessment. Unlike traditional geometric metrics (min Jacobian, aspect ratio), VLM assessment operates on semantic-level understanding of manufacturing feasibility. The NWN problem we identify highlights an important consideration for deploying language models in technical quality control: the model's tendency to use negation when describing the absence of defects creates systematic scoring errors in similarity-based evaluation.

5.4 Automated Boundary Condition Recognition. A key challenge in scaling engineering datasets is that generated geometries must be paired with physically meaningful simulation setups. The automated boundary condition recognition in Stage 3 addresses this by detecting interface regions (load and bolt points) on generated meshes without manual intervention. This automation is essential for scaling from hundreds to tens of thousands of simulated designs, as manual boundary condition assignment would be prohibitively expensive at this scale.

5.5 Limitations and Future Work. Several limitations should be noted:

- VLM classifier accuracy (76.10%, threshold-tuned; 72.17% at the fixed default threshold) leaves room for improvement. Fine-tuning VLMs on domain-specific engineering assessment data could substantially improve filtering quality.
- The 2D-to-3D conversion introduces some geometric artifacts, particularly in thin-wall regions. Post-processing and mesh repair techniques could mitigate these issues.
- The boundary condition recognition module relies on geometric heuristics; learning-based approaches could improve robustness on highly novel topologies.
- While our pipeline demonstrates feasibility with jet engine brackets, validation on additional engineering domains (automotive, aerospace, medical devices) would strengthen generalizability claims.
- We assess augmentation quality intrinsically (manufacturability, label fidelity, and distributional consistency) but do not yet measure its downstream benefit; quantifying how the augmented data improves surrogate or generative models is an important next step.
- Absolute displacement and stress magnitudes are mesh-confounded across datasets—only mass is validated in absolute terms—so the dataset comparison relies on distribution shape rather than absolute values.

Future work will focus on: (1) improving boundary condition recognition with learning-based methods for broader domain applicability, (2) AI-driven autonomous design agents that leverage geometry foundation models (GFMs) for rapid design space exploration, and (3) extension to diverse engineering domains beyond brackets.

6 Conclusion

We have presented DeepJEB++, a large-scale 3D engineering dataset of jet engine brackets constructed by leveraging pretrained foundation models under severely constrained resources. Our three-stage pipeline—2D latent space augmentation with quality-controlled Stable Diffusion interpolation (Stage 1), 3D mesh generation via fine-tuned TRELIS (Stage 2), and automated boundary condition recognition with CAE simulation (Stage 3)—expands 380 seed designs into 15,360 3D geometries with simulation labels, representing a 40 $\times$ scale increase over the original SimJEB dataset.

Key technical contributions include: (1) a foundation-model-driven cross-dimensional (2D $\rightarrow$ 3D) data-augmentation framework for simulation-labeled 3D engineering data, (2) a VLM-based engineering quality filtering framework with the identification and resolution of the NWN problem, (3) domain-specific fine-tuning of

SimJEB vs DeepJEB++ : mass comparison

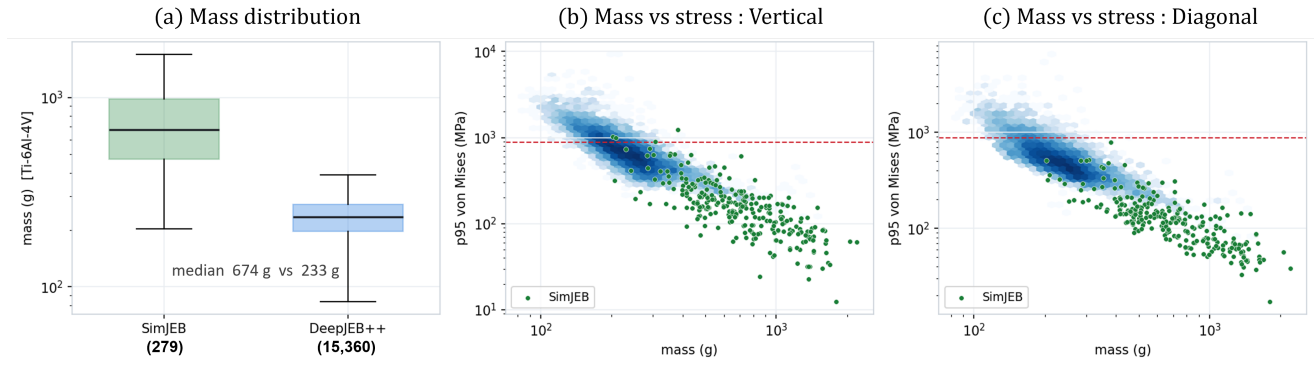


Fig. 13 Mass comparison between SimJEB ($n=279$) and DeepJEB++ ($n=15,360$ deployable). (a) *Mass distribution*: part-mass distributions (Ti-6Al-4V), computed from mesh volume and hence mesh-robust. (b, c) *Mass vs. stress*: 95th-percentile von Mises stress versus mass for the (b) vertical and (c) diagonal load cases (DeepJEB++ as blue point density, SimJEB as green points; the dashed line marks the 903 MPa Ti-6Al-4V yield).

SimJEB vs DeepJEB++ structural-response distribution

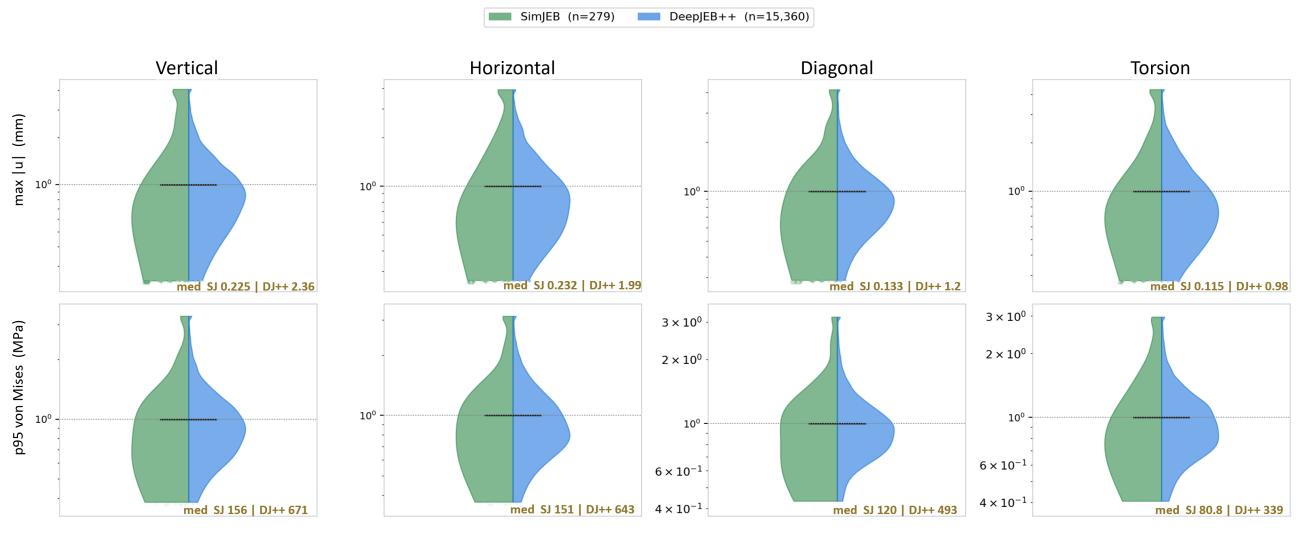


Fig. 14 Self-normalized structural-response distributions for SimJEB ($n=279$) and DeepJEB++ ($n=15,360$ deployable), per load case (columns) and response (rows: maximum displacement $|\mathbf{u}|$, 95th-percentile von Mises stress). Each sample is divided by its own dataset median (dashed line at 1.0), making the comparison robust to the mesh-resolution difference; absolute medians (amber) are mesh-confounded and excluded. Node-level fidelity is established separately in Fig. 10.

a 3D generative foundation model with significant reconstruction quality improvements, and (4) an automated boundary condition recognition and CAE simulation pipeline that produces physics-based performance labels at scale.

By demonstrating that domain-specific GFM pipelines can be constructed with a single GPU in days rather than hundreds of GPUs over weeks, we provide a practical pathway for research labs and small enterprises to build foundation model-driven engineering datasets. Crucially, because the Stage-1–2 augmentation backbone adapts general-purpose foundation models rather than a generator trained from scratch on the target domain, it is domain-agnostic and—unlike domain-specific implicit approaches rebuilt per domain—can in principle be re-targeted to new domains by adapting the labeling stage, a direction we leave to future work. The dataset will be publicly released.

Acknowledgments

This work was supported by grants from the Ministry of Science and ICT (GTL24031-000, N10250154, No.2022-0-00986) and the Ministry of Trade, Industry & Energy (RS-2025-02317327, RS-2025-25444634). The authors thank the KAIST Smart Design Lab members for helpful discussions.

Funding Data

- Ministry of Science and ICT (Grant Nos. GTL24031-000, N10250154, and 2022-0-00986).
- Ministry of Trade, Industry & Energy (Grant Nos. RS-2025-02317327 and RS-2025-25444634).

Conflict of Interest

The authors declare that they have no conflict of interest.

7 Licensing, Attribution, and Accessibility

The DeepJEB++ dataset is a publicly available resource designed to facilitate data-driven research in 3D engineering design, surrogate modeling, and structural performance prediction. It comprises 15,360 simulation-labeled jet engine brackets generated through the integrated foundation-model pipeline of Section 3, pairing each geometry with automatically computed finite-element labels. An overview of the dataset components is provided in Table 5.

Generation Process All bracket geometries were synthesized by adapting pretrained 2D and 3D foundation models—Stable Diffusion v1.5 [38] and TRELIS [20]—through the three-stage pipeline of Section 3. Simulation labels were produced by the automated boundary-condition recognition and linear-elastic finite-element pipeline of Section 3.4, and validated against the SimJEB ground truth (Section 4.3).

Hosting and Compatibility The dataset is publicly hosted on the Hugging Face Hub at <https://huggingface.co/datasets/KAIST-SmartDesignLab/DeepJEB-PP>, released as per-component compressed archives, and is compatible with standard commercial and open-source CAD/CAE platforms as well as Python-based simulation environments. The finite-element volume (tetrahedral) mesh is not distributed explicitly; it is reproducible from the surface mesh using the tetrahedralization settings of Section 3.4.3.

License The DeepJEB++ dataset is a derivative work built upon the SimJEB [6] and DeepJEB [7] datasets, both of which are released under the Open Data Commons Attribution License (ODC-By v1.0). Consistent with these upstream resources, DeepJEB++ is likewise distributed under the Open Data Commons Attribution License (ODC-By v1.0), which permits use, modification, and redistribution—including for commercial purposes—provided that proper attribution is given. The underlying foundation models (TRELIS and Stable Diffusion v1.5) remain governed by their own permissive licenses.

Table 5 Overview of DeepJEB++ dataset components. The mesh, boundary-condition, and field components are distributed as per-component compressed archives (.tar.gz) that extract to the directories listed below.

Category	Path	Description
Surface meshes	1_surface_meshes.tar.gz	15,360 pose-normalized, watertight triangle meshes (.obj).
Boundary conditions	2_boundary_conditions.tar.gz	15,360 beam-bolt and loaded-clevis interfaces (vertex indices and millimeter frame) per bracket.
Per-load fields	3_fea_fields.tar.gz	Nodal displacement $ u $ and von Mises stress fields, four load cases (2×4 per bracket).
Scalar labels	deepjebpp_labels.csv	Per-load maximum displacement and 95th-percentile von Mises stress.
Mass	deepjebpp_labels.csv	Part mass (enclosed volume $\times$ Ti-6Al-4V density), from the volume-to-mass pipeline validated to $R^2=0.9999$ against CAD on the SimJEB reference set.
Metadata	metadata.json	Material (Ti-6Al-4V), the four load-case magnitudes/directions, and units.

Availability and Maintenance The dataset will remain publicly accessible for at least ten years following its release. Any updates, revisions, or corrections will be announced with appropriate version control and a revised access link. A README file is included to assist users in navigating the dataset.

References

- [1] Oh, S., Jung, Y., Kim, S., Lee, I., and Kang, N. Deep generative design: Integration of topology optimization and generative models. *ASME J. Mech. Des.*, 141(11):111405, 2019.
- [2] Regenwetter, L., Nobari, A. H., and Ahmed, F. Deep generative models in engineering design: A review. *ASME J. Mech. Des.*, 144(7):071704, 2022.
- [3] Nie, Z., Lin, T., Jiang, H., and Kara, L. B. TopologyGAN: Topology optimization using generative adversarial networks based on physical fields over the initial domain. *ASME J. Mech. Des.*, 143(3):031715, 2021.
- [4] Cunningham, J. D., Simpson, T. W., and Tucker, C. S. An investigation of surrogate models for efficient performance-based decoding of 3D point clouds. *ASME J. Mech. Des.*, 141(12):121401, 2019.
- [5] Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., and Battaglia, P. W. Learning mesh-based simulation with graph networks. *ICLR*, 2021.
- [6] Whalen, E., Beyene, A., and Mueller, C. SimJEB: Simulated jet engine bracket dataset. *Computer Graphics Forum*, 40(5):9–17, 2021.
- [7] Hong, S., Kwon, Y., Shin, D., Park, J., and Kang, N. DeepJEB: 3D deep learning-based synthetic jet engine bracket dataset. *ASME J. Mech. Des.*, 147(4):041703, 2025.
- [8] Park, J. J., Florence, P., Straub, J., Newcombe, R., and Lovegrove, S. DeepSDF: Learning continuous signed distance functions for shape representation. *CVPR*, 2019.
- [9] PhysicsX. Building beyond human imagination with foundation models for geometry and physics. <https://www.physicsx.ai/newsroom/>, 2024.
- [10] Chang, A. X., Funkhouser, T., Guibas, L., et al. ShapeNet: An information-rich 3D model repository. *arXiv preprint arXiv:1512.03012*, 2015.
- [11] Koch, S., Matveev, A., Jiang, Z., et al. ABC: A big CAD model dataset for geometric deep learning. *CVPR*, 2019.
- [12] Regenwetter, L., Weaver, C., and Ahmed, F. FRAMED: An AutoML approach for structural performance prediction of bicycle frames. *Computer-Aided Design*, 156, 2023.
- [13] Regenwetter, L., Curry, B., and Ahmed, F. BIKED: A dataset for computational bicycle design with machine learning benchmarks. *ASME J. Mech. Des.*, 144(3):031706, 2022.
- [14] Bagazinski, N. J. and Ahmed, F. Ship-D: Ship hull dataset for design optimization using machine learning. *IDETC/CIE*, 2023.
- [15] Elrefaie, M., Morar, F., Dai, A., and Ahmed, F. DriveAerNet++: A large-scale multimodal car dataset with CFD simulations and deep learning benchmarks. *NeurIPS*, 2024.
- [16] Cobb, A. D., et al. AircraftVerse: A large-scale multimodal dataset of aerial vehicle designs. *NeurIPS Datasets and Benchmarks Track*, 2023.
- [17] Nichol, A., Jun, H., Dhariwal, P., Mishkin, P., and Chen, M. Point-E: A system for generating 3D point clouds from complex prompts. *arXiv:2212.08751*, 2022.
- [18] Liu, R., Wu, R., Van Hoorick, B., Tokmakov, P., Zakharov, S., and Vondrick, C. Zero-1-to-3: Zero-shot one image to 3D object. *ICCV*, 2023.
- [19] Poole, B., Jain, A., Barron, J. T., and Mildenhall, B. DreamFusion: Text-to-3D using 2D diffusion. *ICLR*, 2023.
- [20] Xiang, J., Lv, Z., Xu, S., Deng, Y., Wang, R., Zhang, B., Chen, D., Tong, X., and Yang, J. Structured 3D latents for scalable and versatile 3D generation (TRELIS). *CVPR*, 2025.

- [21] Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., and Ng, R. NeRF: Representing scenes as neural radiance fields for view synthesis. *ECCV*, 2020.
- [22] Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4), 2023.
- [23] Liu, X., Gong, C., and Liu, Q. Flow straight and fast: Learning to generate and transfer data with rectified flow. *ICLR*, 2023.
- [24] Hong, Y., Zhang, K., Gu, J., Bi, S., Zhou, Y., Liu, D., Liu, F., Sunkavalli, K., Bui, T., and Tan, H. LRM: Large reconstruction model for single image to 3D. *ICLR*, 2024.
- [25] Liu, M., Xu, C., Jin, H., Chen, L., Varma T. M., Xu, Z., and Su, H. One-2-3-45: Any single image to 3D mesh in 45 seconds without per-shape optimization. *NeurIPS*, 2023.
- [26] Long, X., Guo, Y.-C., Lin, C., Liu, Y., Dou, Z., Liu, L., Ma, Y., Zhang, S.-H., Habermann, M., Theobalt, C., and Wang, W. Wonder3D: Single image to 3D using cross-domain diffusion. *CVPR*, 2024.
- [27] Xu, J., Cheng, W., Gao, Y., Wang, X., Gao, S., and Shan, Y. InstantMesh: Efficient 3D mesh generation from a single image with sparse-view large reconstruction models. *arXiv:2404.07191*, 2024.
- [28] Li, Y., Zou, Z.-X., Liu, Z., Wang, D., Liang, Y., Yu, Z., Liu, X., Guo, Y.-C., Liang, D., Ouyang, W., and Cao, Y.-P. TripoSG: High-fidelity 3D shape synthesis using large-scale rectified flow models. *arXiv:2502.06608*, 2025.
- [29] Tencent Hunyuan3D Team. Hunyuan3D 2.0: Scaling diffusion models for high-resolution textured 3D assets generation. *arXiv:2501.12202*, 2025.
- [30] Chen, W. and Ahmed, F. PaDGAN: Learning to generate high-quality novel designs. *ASME J. Mech. Des.*, 143(3):031703, 2021.
- [31] Heyrani Nobari, A., Chen, W., and Ahmed, F. PcDGAN: A continuous conditional diverse generative adversarial network for inverse design. *KDD*, 2021.
- [32] Shu, D., Cunningham, J., Stump, G., et al. 3D design using generative adversarial networks and physics-based validation. *ASME J. Mech. Des.*, 142(7):071701, 2020.
- [33] Wang, J., Chen, W. W., Da, D., Fuge, M., and Rai, R. IH-GAN: A conditional generative model for implicit surface-based inverse design of cellular structures. *CMAME*, 396:115060, 2022.
- [34] Mazé, F. and Ahmed, F. Diffusion models beat GANs on topology optimization. *Proc. AAAI Conf. Artif. Intell.*, 37(8):9108–9116, 2023.
- [35] Radford, A., Kim, J. W., Hallacy, C., et al. Learning transferable visual models from natural language supervision. *ICML*, 2021.
- [36] Li, J., Li, D., Xiong, C., and Hoi, S. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. *arXiv:2301.12597*, 2023.
- [37] Liu, H., Li, C., Wu, Q., and Lee, Y. J. Visual instruction tuning. *arXiv:2304.08485*, 2023.
- [38] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. *CVPR*, 2022.
- [39] Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *NeurIPS*, 2020.
- [40] Wang, C. and Golland, P. Interpolating between images with diffusion models. *ICML*, 2023.
- [41] Caron, M., Touvron, H., Misra, I., et al. Emerging properties in self-supervised vision transformers. *ICCV*, 2021.
- [42] Kasa, I. A circle fitting procedure and its error analysis. *IEEE Trans. Instrum. Meas.*, 25(1):8–14, 1976.
- [43] Umeyama, S. Least-squares estimation of transformation parameters between two point patterns. *IEEE Trans. Pattern Anal. Mach. Intell.*, 13(4):376–380, 1991.
- [44] Zhao, W., Bai, L., Rao, Y., Zhou, J., and Lu, J. UniPC: A unified predictor-corrector framework for fast sampling of diffusion models. *NeurIPS*, 2023.
- [45] Ho, J. and Salimans, T. Classifier-free diffusion guidance. *arXiv:2207.12598*, 2022.